

Kỹ Thuật Lập Trình Python

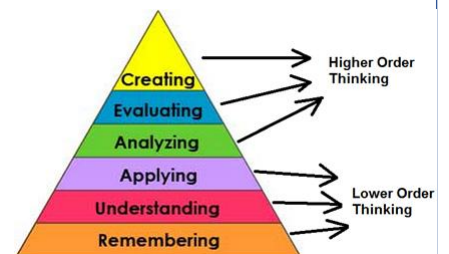
MODULE - PACKAGE



1



- Vận dụng module quản lý các hàm, biến số...
- Áp dụng tổ chức project bằng package.



2



Nội dung

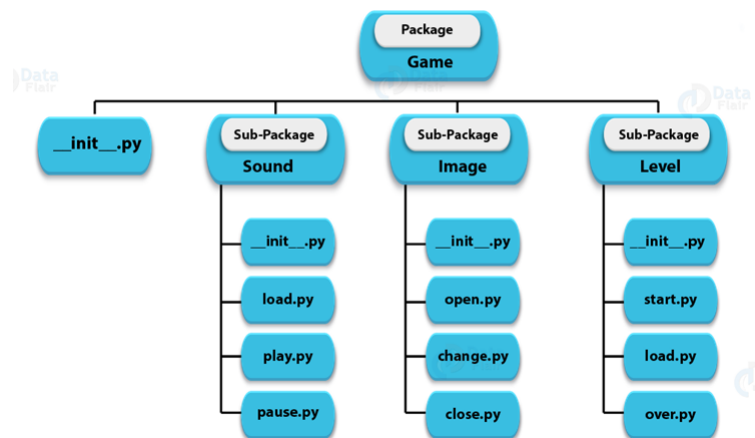
1. Module
2. Package

3



Giới thiệu cấu trúc chung của Module và Package

Package Module Structure



4



module

Giới thiệu module

- Là một file lưu trữ mã nguồn (**source code**) của Python, được lưu với dạng **<tên_file.py>**.
- module (file) chứa:
 - biến,
 - câu lệnh,
 - hàm,
 - Class
 - ...
- Ý tưởng dùng mô-đun là để **chia nhỏ** các chương trình lớn thành các file nhỏ để dễ dàng quản lý,
- Dễ dàng tối ưu các đoạn mã code nếu cần.



module

Giới thiệu module

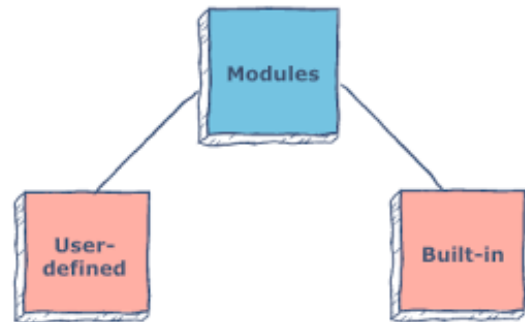
- Sử dụng module khi chương trình có phạm vi thực hiện lớn, **tránh việc lưu trữ mã nguồn chỉ duy nhất một file**.
- Chia nhỏ thành từng **tính năng, nhiệm vụ**, và phải được lưu trữ chúng thành các module.
- module có source file có đuôi là **.py**



module

Có hai loại module

- Standard module (**Built-in**)
- Module tự định nghĩa (**User-defined**)



Đọc thêm ref.: <https://data-flair.training/blogs/python-modules/>

Kỹ thuật Lập trình Python

7

7



module

Có hai loại module

- **Standard module**: là các module chuẩn, có sẵn/mặc định trong Python.
- Ví dụ như: math, random...
- Các module chuẩn có sẵn trong Python:
<https://docs.python.org/3/py-modindex.html>
- **Module tự định nghĩa**: là các module tự tạo để lưu trữ các mã nguồn.

Kỹ thuật Lập trình Python

8

8



module

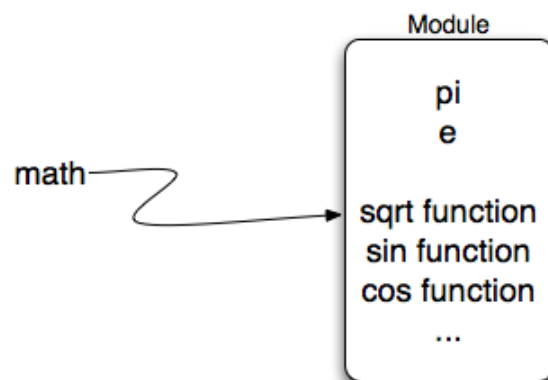
Danh sách các built-in module

- Ref: <https://docs.python.org/3/py-modindex.html>



module

Ví dụ





module

Sử dụng built-in module bằng cách nào?

- Sử dụng từ khóa **import** để trỏ đến các module

Minh họa việc import một mô-đun có sẵn:

```
import math as m
import random as r
```



module có sẵn

Cú pháp import mô-đun có sẵn

- `import name_module1, name_module2..., module_n`

Cách gọi hàm từ một module

- `<name_module>.<function>`



module có sẵn

Ví dụ

```
import math, random
print("Tính Cos(30) =", math.cos(30))
print("Random một giá trị ngẫu nhiên", random.random())
```

Kết quả:

```
Tính Cos(30) = 0.15425144988758405
Random một giá trị ngẫu nhiên 0.9148759360621969
```



module có sẵn

- Từ khóa **as** qui định tên định danh mới cho mô-đun.
- Ví dụ:

```
import math as m, random as r
print("Tính Cos(30) =", m.cos(30))
print("Random một giá trị ngẫu nhiên", r.random())
```



module có sẵn

Cú pháp *from...import*

- Cú pháp “**from ... import**” để sử dụng một hàm nào đó cụ thể trong mô-đun

from module **import** function1, function2, function3,...

- Hoặc, **from** module **import *** (sử dụng tất cả những gì có trong module và không import các hàm bắt đầu bằng ký tự “_”).



module tự định nghĩa

GV minh họa

- Minh họa trong PyCharm



package

package?

- Là folder lưu trữ các mô-đun code.
- Nếu chương trình có nhiều mô-đun liên quan nhau thì nên được quản lý cùng một Package
- Trong folder có một file tên là `__init__.py`



package

Cú pháp import package

- Dùng import


```
import package_name
import package_name.module_name
```
- Dùng from:


```
from package_name.module_name import function_name...
from package_name import module_names
```



package

Ví dụ

```
from illustrating_package import illustrating_module as im
im.printing_pos_UPPER(["trường", "đại", "học", "công",
"nghệ", "thông", "tin"])
```

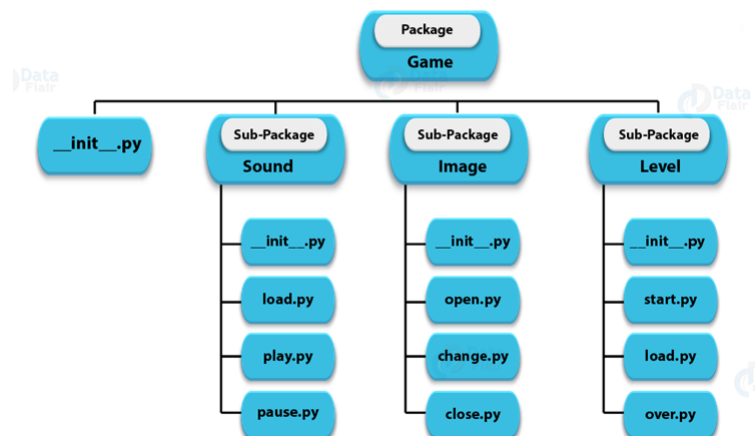
Kết quả:

```
[(0, 'TRƯỜNG'), (1, 'ĐẠI'), (2, 'HỌC'), (3, 'CÔNG'), (4,
'NGHỆ'), (5, 'THÔNG'), (6, 'TIN')]
```



Tóm tắt

Package Module Structure





Phân biệt thư viện – module – package

Thư viện - library

- Thư viện trong Python chứa các mô-đun và package, được thiết kế cung cấp các chức năng, công cụ để giải quyết vấn đề cụ thể.

Ví dụ

- Thư viện NumPy là một thư viện hỗ trợ tính toán khoa học và số học.

Cách kiểm tra

- Câu lệnh `pip list` để kiểm tra một thư viện có tồn tại



Phân biệt thư viện – module – package

Mô-đun - module

- Mô-đun là tập tin chứa mã Python dùng để định nghĩa biến, hàm và lớp.

Ví dụ

- Mô-đun math chứa các hàm, hằng số toán học như sin, cos, và pi...

Cách kiểm tra

- Câu lệnh `help()` hoặc `dir()` để kiểm tra các mô-đun.

```
print(help('math'))
print(dir('random'))
```



Phân biệt thư viện – module – package

Gói - package

- Thư mục chứa các mô-đun liên quan.
- Tổ chức và nhóm các mô-đun có chức năng tương tự lại với nhau.

Ví dụ

- Gói numpy là một gói phổ biến trong thư viện NumPy.

Cách kiểm tra

- `dir()` liệt kê tất cả các thành phần (bao gồm mô-đun, gói con và biến) có sẵn trong package đó.
- `pip show package_name`





Hỏi - Đáp

